

Local Model based development

Local models can be used to ensure full privacy and eliminate need for API based access of cloud hosted LLMs, which are usually charged per token.

Two popular tools are available for this:

- LM Studio
- Ollama

This document will walk you through setting up these tools (either one is fine, LM Studio is more comprehensive and GUI based) for local development.

LM Studio

Use this walkthrough to download **LM Studio**, load the *llama-3.2-3b-instruct* model, and expose an OpenAI-compatible endpoint (`http://127.0.0.1:1234/v1`) so the sample Python script runs without edits.

0. Prerequisites

- **Hardware:** 8 GB RAM (min), 4-core CPU; discrete GPU (NVIDIA $\geq$ 6 GB VRAM) strongly recommended for speed.
- **OS:** Windows 10 +, macOS 12 +, Ubuntu 20.04 + (or any distro that supports AppImage).
- **Python:** 3.8 + and pip.

1. Download LM Studio

OS	Download link	File type
Windows	https://lmstudio.ai → <i>Download for Windows</i>	`` installer
macOS	https://lmstudio.ai → <i>Download for macOS</i>	`` image
Linux	“Other Platforms” → <i>AppImage / .deb</i>	or

Tip – If you prefer command line, Linux builds are also published on GitHub Releases: `wget https://github.com/lmstudio-public/lmstudio/releases/download/vX.Y.Z/lmstudio-X.Y.Z.AppImage`.

2. Install & First Launch

1. **Windows** – run the .exe, accept defaults.
2. **macOS** – open the .dmg and drag *LM Studio* into **Applications**.
3. **Linux** –

```
chmod +x lmstudio-X.Y.Z.AppImage
./lmstudio-X.Y.Z.AppImage
```

When LM Studio opens the first time it asks for a models directory; keep the default or point to a dedicated SSD folder.

3. Download the Model

1. Navigate to **Models** ▶ **Download Models**.
2. In the search bar type “**llama-3.2-3b-instruct**” (or similar 3 B parameter variant).*
If you can’t find the exact name, choose the closest 3 B “-instruct” checkpoint – LM Studio exposes whatever you download as that model name.
3. Click **Download**. The progress shows in the bottom panel. Wait until the green **Ready** badge appears.

Quantization note: 4-bit Q4_0 is memory-friendly; leave default unless you have GPU VRAM to spare.

4. Enable the API Server

1. Open **Settings** ▶ **API Server** (left sidebar).
 2. Toggle “**Enable OpenAI-Compatible API Server**” *on*.
 3. Set **Bind Address** → 127.0.0.1
Port → 1234
Base Path → /v1
 4. Under **Model Mapping**, click **Add** and pick *llama-3.2-3b-instruct* → save.
 5. Press **Start Server**. Status bar turns green: `http://127.0.0.1:1234/v1`.
-

5. Verify the Endpoint

```
curl http://127.0.0.1:1234/v1/models
```

Should return a JSON listing with your model.

You can also send a quick chat request:

```
curl http://127.0.0.1:1234/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "llama-3.2-3b-instruct", "messages": [{"role": "user", "content": "Hello!"}]}'
```

6. Install the Python Client

```
# Newer openai ≥1.0 uses updated method names
# LM Studio's server follows the 0.x spec, so pin 0.28.1
pip install "openai<1.0.0" # installs 0.28.1 today
```

If you already have openai ≥1.0, uninstall first: `pip uninstall openai`.

7. Run the Sample Script

Save the snippet provided (e.g., `lmstudio_test.py`) and execute:

```
python lmstudio_test.py
```

Expected output (trimmed):

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "The capital of France is Paris."
      }
    }
  ]
}
```

8. Troubleshooting

Symptom	Fix
``	Make sure API Server toggle is on , port 1234 not blocked by firewall, and LM Studio is still running.
``	You are using openai ≥1.0. Downgrade to <code>pip install openai==0.28.1</code> .
High VRAM	Switch model quantization to 4-bit or 8-bit: Models ▶ ... ▶ Download

Symptom	Fix
usage / OOM	<i>Options.</i>
Slow responses	Enable GPU in Settings ▶ Hardware Acceleration ; or load a smaller model.

9. Optional – Run Server from CLI

LM Studio bundles `lm-studio-server`.

```
lm-studio-server \
  --model-path "~/LMStudio/models/llama-3.2-3b-instruct.Q4_0.gguf" \
  --port 1234 --host 127.0.0.1
```

Good for headless servers or systemd services.

10. Next Steps

- Try bigger models (e.g., Llama-3-8B-Instruct) by repeating step 3.
- Integrate with LangChain: set `llm = ChatOpenAI(openai_api_base="http://127.0.0.1:1234/v1", model_name="llama-3.2-3b-instruct")`.
- Protect the endpoint with a reverse proxy & auth when exposing beyond localhost.

Happy local inferencing!

Ollama Local API Setup Guide

Run open-source models with **Ollama** and expose an OpenAI-compatible endpoint so the **same Python snippet** (or a tiny port edit) just works.

0. Prerequisites

- **CPU/GPU:** Similar to LM Studio; GPU (6 GB VRAM +) gives large speed-ups.
- **OS:** Windows 10 +, macOS 12 +, any modern Linux.
- **CLI:** `curl`, `git`, and **Python 3.8 +**.

1. Install Ollama

OS	Command / Download
macOS	<code>brew install ollama</code>

OS	Command / Download
Windows	Get the Ollama-Setup.msi from https://ollama.com/download and run it
Linux	```bash
curl -fsSL	sh
https://ollama.com/install.sh	

After install, open a new terminal so `ollama` is in your `PATH`.

2. Start the API Server

Ollama bundles its own server; simply run:

```bash

ollama serve # listens on http://127.0.0.1:11434 by default

Using port **1234** (to keep the sample code unchanged)

*# macOS / Linux*

export OLLAMA\_HOST=127.0.0.1:1234

ollama serve

*# Windows PowerShell*

set OLLAMA\_HOST=127.0.0.1:1234

ollama serve

Ollama now exposes `` with full OpenAI-compatibility – the exact base URL your script expects.([reddit.com](https://reddit.com))

---

## 3. Download a Model

Pull any model you like (e.g., the Llama-3 8 B instruct checkpoint):

ollama pull llama3:8b-instruct

**Alias tip** – If you want to keep the model name in the Python sample (llama-3.2-3b-instruct), create a lightweight alias:

```
ollama create llama-3.2-3b-instruct --modelfile - <<'EOF'
FROM llama3:8b-instruct
EOF
```

---

## 4. Verify the Endpoint

curl http://127.0.0.1:1234/v1/models

You should see your newly pulled (or aliased) model in the JSON.

Quick chat check:

```
curl http://127.0.0.1:1234/v1/chat/completions \
 -H "Content-Type: application/json" \
 -d '{"model":"llama-3.2-3b-instruct", "messages":[{"role":"user", "content":"Hello, Ollama!"}]}'
```

---

## 5. Python Client

Ollama's endpoint follows the *newer* OpenAI spec, so you may use the latest `openai` ( $\geq 1.0$ ) or the same `0.28.1` pinned earlier. For consistency you can leave the import exactly as in the original snippet.

Just ensure that `openai.api_base` points to `http://127.0.0.1:1234/v1` (or `11434` if you skipped the port change).

---

## 6. Troubleshooting

Symptoms  
Resolution

---

- `` Confirm `ollama serve` is running and firewall isn't blocking port `1234/11434`.
  - `` Newer `openai` library installed *while* you left server on 0-series spec – upgrade Ollama ( $\geq$  Feb 2024) or pin `openai 0.28.1`.
  - `` Another Ollama instance already running. End it (`pkill ollama` / Task Manager) or choose a different `OLLAMA_HOST` port.
- 

## 7. Next Steps

- Experiment with other models: `ollama pull mistral:7b-instruct`, `ollama pull codellama`.
- Combine Ollama & LM Studio behind an Nginx reverse proxy with path-based routing for multiple local LLMs.
- Use in LangChain: `ChatOpenAI(openai_api_base='http://127.0.0.1:1234/v1', model_name='llama-3.2-3b-instruct')`.

*That's it – you now have two local engines (LM Studio and Ollama) that speak the same API!*